

Spring Boot

It is as an open-source, Java-based framework used to build standalone applications with minimal configurations.

Standalone - The application is completely **self-contained**, independent, and can run on its **own without relying** on an **externally** installed web server or application server

Before Spring:

- Developers had to create and manage objects **manually**.
- Large amounts of **boilerplate code**.
- **Tight coupling** between classes.

Spring solves these problems using:

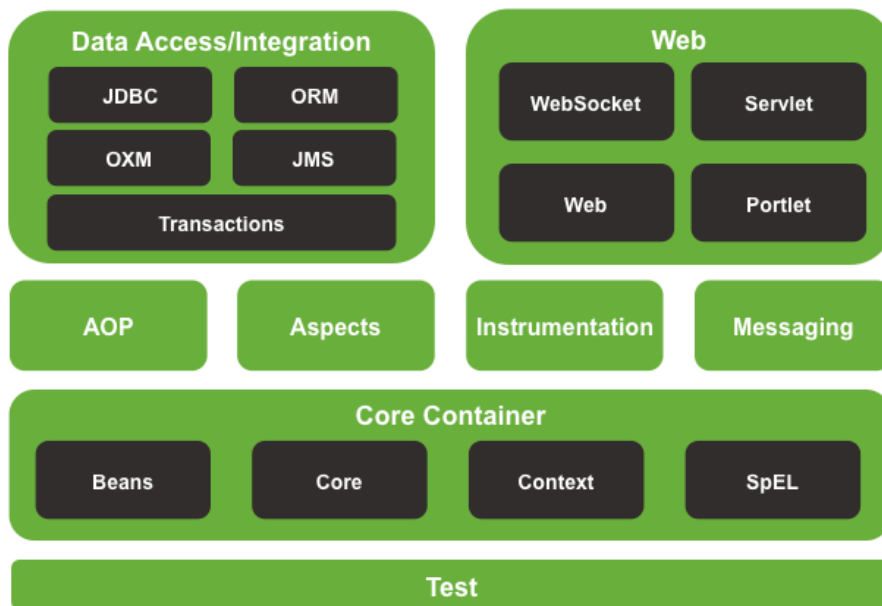
- IoC (Inversion of Control)
- DI (Dependency Injection)
- Modular architecture
- Easy database integration
- Easy web application development

Example Applications are - Banking applications, E-commerce applications.

Spring boot runtime



Spring Framework Runtime



Spring Boot runtime behavior relies on an embedded web server running on a Java Virtual Machine (JVM) or executing as a native binary via GraalVM

1. Core Container

a. Core and Beans

Provide **fundamental parts** (spring-core and spring-beans) of the framework, including IoC and Dependency Injection features. **To handle object creation, wiring and lifecycle management.**

It removes the need of programmatic singletons and allows you to decouple the configuration and specification of dependencies.

Beans - manages Java objects, provide standard BeanFactory interface , handles initialization, configuration and property manipulation.

b. Context

Builds on solid base provided by core and beans to add **enterprise-level services.**

It represents the runtime of IoC container.

manages features like internationalization, event propagation and resource loading.

Internationalization - To **design and structure** your application . so, It can adapt **automatically** to different languages **without changing any source code**.

c. **Spring Expression Language (SpEL)**

Acts as an expression mechanism to manipulate objects at runtime.

Function - It allows developers to **query objects**

2. **Data Access/integration**

a. **JDBC (Java Database Connections)**

Provides abstraction-layer that removes to do tedious **JDBC coding** and parsing database-vendor.

b. **ORM (Object Relational Mapping)**

Provides **Integration layers**, that integrates external **data persistence engines** like **Hibernate, JPA, JDO, iBatis** with spring dependency injection and transaction management.

c. **OXM (Object XML mapping)**

Designed to ease the conversion between java object and XML.

1. **Marshalling (Serialization)** : Converting a Java object graph into an XML document.

2. **Unmarshalling (Deserialization)** : Converting an XML document back into a Java object graph

d. **Java Messaging service (JMS)**

features for producing/consuming of messages.

e. **Transaction**

Supports **programmatic** and **declarative transaction management** for classes that implement **special interfaces** and for *all your POJOs (plain old Java objects)*.

